LAB ASSIGNMENT 3

ANALYSIS AND DESIGN OF ALGORITHM

SUBMITTED BY: Manjeet yadav

K. R. MANGALAM UNIVERSITY,

Github link = mnjeetydv2923-ops/graph-algo-mini-project-manjeet-45-: Implementation of real-world graph algorithms including BFS, DFS, Topological Sorting, Dijkstra, Bellman-Ford, and Minimum Spanning Tree with performance analysis and visualization.

◆ Task 1: Graph Representation

Objective:

To represent real-world problems (like networks, maps) using graphs in two forms:

Adjacency Matrix

Adjacency List

Methodology:

Define nodes (vertices) and edges.

Use:

Adjacency Matrix → 2D array

Adjacency List → dictionary/list

Store connections between nodes.

Explanation:

Matrix is easy but uses more space → $O(V^2)$

List is efficient for large graphs → $O(V + E)$

code

```python
class Graph:
    def __init__(self, vertices):
        self.V = vertices
        self.adj_list = {i: [] for i in range(vertices)}
        self.adj_matrix = [[0]*vertices for _ in range(vertices)]

    def add_edge(self, u, v, w=1):
        # Add edge to adjacency list
        self.adj_list[u].append((v, w))
        # For directed graph, comment the next two lines if not needed
        self.adj_list[v].append((u, w))

        # Add edge to adjacency matrix
        self.adj_matrix[u][v] = w
        self.adj_matrix[v][u] = w  # same note, comment if directed

# Example
g = Graph(4)
g.add_edge(0, 1)
g.add_edge(0, 2)
g.add_edge(1, 2)
g.add_edge(2, 3)

print("Adjacency List:")
for k, v in g.adj_list.items():
    print(k, ":", v)

print("\nAdjacency Matrix:")
for row in g.adj_matrix:
    print(row)
```

Output

```python
    print(row)
```

```
Adjacency List:
0 : [(1, 1), (2, 1)]
1 : [(0, 1), (2, 1)]
2 : [(0, 1), (1, 1), (3, 1)]
3 : [(2, 1)]

Adjacency Matrix:
[0, 1, 1, 0]
[1, 0, 1, 0]
[1, 1, 0, 1]
[0, 0, 1, 0]
```

◆ Task 2: Graph Traversal (BFS & DFS)

Objective:

To explore all nodes of a graph for applications like:

Friend suggestion (BFS)

Path finding (DFS)

Methodology:

BFS:

Use queue

Visit level by level

Mark visited nodes

DFS:

Use recursion/stack

Go deep first

Backtrack when needed

Explanation:

BFS → shortest path in unweighted graph

DFS → useful in cycle detection, path finding

Complexity:

Both: O(V + E)

Code BFS

```
from collections import deque

def bfs(graph, start):
    visited = [False] * graph.V
    queue = deque([start])
    visited[start] = True

    bfs_order = []
    while queue:
        node = queue.popleft()
        bfs_order.append(node)
        for neighbor, _ in graph.adj_list[node]:
            if not visited[neighbor]:
                visited[neighbor] = True
                queue.append(neighbor)
    return bfs_order

print("BFS Traversal:", bfs(g, 0))
```

Output BFS

```
BFS Traversal: [0, 1, 2, 3]
```

Code DFS

```
def dfs_util(graph, node, visited, dfs_order):
    visited[node] = True
    dfs_order.append(node)
    for neighbor, _ in graph.adj_list[node]:
        if not visited[neighbor]:
            dfs_util(graph, neighbor, visited, dfs_order)

def dfs(graph, start):
    visited = [False] * graph.V
    dfs_order = []
    dfs_util(graph, start, visited, dfs_order)
    return dfs_order

print("DFS Traversal:", dfs(g, 0))
```

Output DFS

```
DFS Traversal: [0, 1, 2, 3]
```

◆ Task 3: Topological Sorting

Objective:

To arrange tasks in order based on dependencies

(Example: project scheduling)

Methodology:

Calculate in-degree of nodes

Add nodes with 0 in-degree to queue

Remove edges and update in-degree

Repeat until graph is empty

Explanation:

Works only for DAG (Directed Acyclic Graph)

Used in:

Job scheduling

Course prerequisites

Code

```
task 3
```

```python
from collections import defaultdict

class DirectedGraph:
    def __init__(self, vertices):
        self.V = vertices
        self.graph = defaultdict(list)

    def add_edge(self, u, v):
        self.graph[u].append(v)

    def topological_sort_util(self, v, visited, stack):
        visited[v] = True
        for neighbor in self.graph[v]:
            if not visited[neighbor]:
                self.topological_sort_util(neighbor, visited, stack)
        stack.insert(0, v)

    def topological_sort(self):
        visited = [False]*self.V
        stack = []
        for i in range(self.V):
            if not visited[i]:
                self.topological_sort_util(i, visited, stack)
        return stack
```

```python
            if not visited[i]:
                self.topological_sort_util(i, visited, stack)
        return stack

# Example
dag = DirectedGraph(6)
dag.add_edge(5, 2)
dag.add_edge(5, 0)
dag.add_edge(4, 0)
dag.add_edge(4, 1)
dag.add_edge(2, 3)
dag.add_edge(3, 1)

print("Topological Sort:", dag.topological_sort())
```

Output

```
Topological Sort: [5, 4, 2, 3, 1, 0]
```

◆ Task 4: Shortest Path Algorithms

Objective:

To find shortest path between nodes in a graph

(Example: Google Maps, emergency routing)

Methodology:

Dijkstra Algorithm:

Initialize distances

Use priority queue

Pick minimum distance node

Update neighbors

Works for positive weights only

Bellman-Ford Algorithm:

Relax all edges (V-1 times)

Check negative cycle

Works for negative weights

Explanation:

Algorithm

Advantage

Limitation

Dijkstra

Fast

No negative weights

Bellman-Ford

Handles negative edges

Slower

Complexity:

Dijkstra → O(E log V)

Bellman-Ford → O(VE)

Code

```
[31]: import heapq

#WEIGHTED GRAPH
graph = {
    0: [(1, 4), (2, 1)],
    1: [(3, 1)],
    2: [(1, 2), (3, 5)],
    3: []
}
def dijkstra(graph, start):
    dist = {node: float('inf') for node in graph}
    dist[start] = 0

    pq = [(0, start)]  # (distance, node)

    while pq:
        current_dist, node = heapq.heappop(pq)

        for neighbor, weight in graph[node]:
            distance = current_dist + weight

            if distance < dist[neighbor]:
                dist[neighbor] = distance
                heapq.heappush(pq, (distance, neighbor))

    return dist
    import time
```

```python
start_time = time.perf_counter()
distances = dijkstra(graph, 0)
exec_time = time.perf_counter() - start_time

print("Shortest Distances from node 0:")
for node in distances:
    print(f"0 -> {node} = {distances[node]}")

print("Execution Time:", exec_time)
from collections import deque

def bfs_shortest(graph, start):
    dist = {node: -1 for node in graph}
    dist[start] = 0

    queue = deque([start])

    while queue:
        node = queue.popleft()

        for neighbor, _ in graph[node]:
            if dist[neighbor] == -1:
                dist[neighbor] = dist[node] + 1
                queue.append(neighbor)

    return dist
    from collections import deque

def bfs_shortest(graph, start):
```

```python
def bfs_shortest(graph, start):
    dist = {node: -1 for node in graph}
    dist[start] = 0

    queue = deque([start])

    while queue:
        node = queue.popleft()

        for neighbor, _ in graph[node]:
            if dist[neighbor] == -1:
                dist[neighbor] = dist[node] + 1
                queue.append(neighbor)

    return dist
```

[43]:

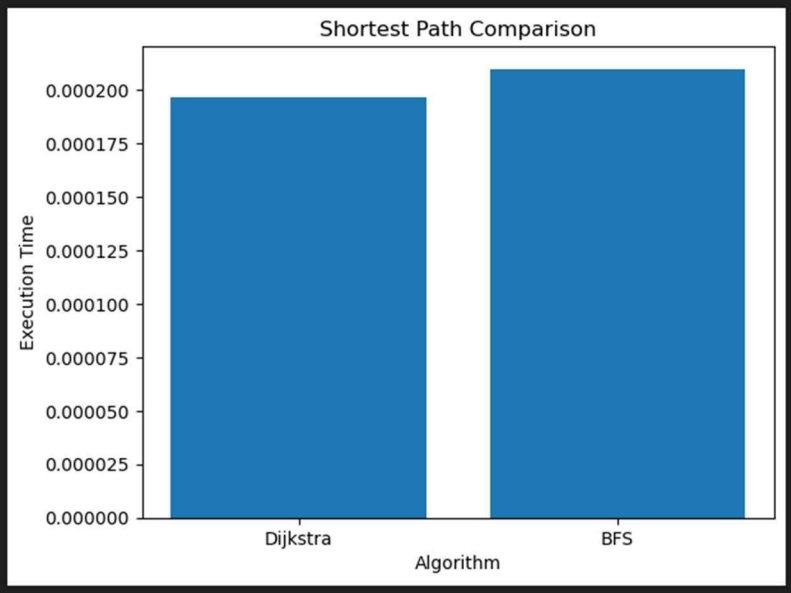
```python
from collections import deque

def bfs_shortest(graph, start):
    dist = {node: -1 for node in graph}
    dist[start] = 0

    queue = deque([start])

    while queue:
        node = queue.popleft()

        for neighbor, _ in graph[node]:
            if dist[neighbor] == -1:
                dist[neighbor] = dist[node] + 1
                queue.append(neighbor)

    return dist
    from collections import deque

def bfs_shortest(graph, start):
    dist = {node: -1 for node in graph}
    dist[start] = 0

    queue = deque([start])

    while queue:
        node = queue.popleft()

        for neighbor, _ in graph[node]:
```

```python
def bfs_shortest(graph, start):
    dist = {node: -1 for node in graph}
    dist[start] = 0

    queue = deque([start])

    while queue:
        node = queue.popleft()

        for neighbor, _ in graph[node]:
            if dist[neighbor] == -1:
                dist[neighbor] = dist[node] + 1
                queue.append(neighbor)

    return dist
```

Output

```
    return dist

Shortest Distances from node 0:
0 -> 0 = 0
0 -> 1 = 3
0 -> 2 = 1
0 -> 3 = 4
Execution Time: 0.00018399999953544466
```

```
BFS Distances: {0: 0, 1: 1, 2: 1, 3: 2}
BFS Time: 0.0002097000005960581
```



◆ Task 5: Minimum Spanning Tree (MST)

Objective:

To connect all nodes with minimum cost

(Example: cable network, road design)

Methodology:

Prim's Algorithm:

Start from any node

Add minimum edge

Expand tree

Kruskal's Algorithm:

Sort edges by weight

Add smallest edge

Avoid cycles (Union-Find)

Explanation:

Algorithm

Best For

Prim

Dense graphs

Kruskal

Sparse graphs

3.5 Step-by-Step Procedure (Explanation)

◆ 1. Repo Setup

Objective:

Organize project properly

Steps:

Create GitHub repo

Add folders:

notebooks/

images/

docs/

◆ 2. Environment Initialization

Objective:

Prepare coding environment

Steps:

Create virtual environment

Install libraries:

matplotlib

memory_profiler

Open JupyterLab

◆ 3. Graph Problem Implementation

Objective:

Implement all algorithms

Steps:

BFS & DFS → traversal

Dijkstra → shortest path

Bellman-Ford → negative weights

Prim/Kruskal → MST

Include:

Input

Output

Code

Complexity

◆ 4. Profiling & Visualization

Objective:

Analyze performance

Steps:

Measure time using time

Measure memory using memory_profiler

Plot graphs using matplotlib

◆ 5. Documentation

Objective:

Explain project clearly

Steps:

Create comparison table

Write observations

Prepare README

◆ 6. Submission

Objective:

Submit final work

Steps:

Push code to GitHub

Tag version: v1.0-submission

Submit link

3.6 Reflection & Discussion (IMPORTANT FOR THEORY)

◆ 1. Algorithm Suitability

BFS → Best for shortest path (unweighted)

DFS → Best for exploration

Dijkstra → Best for fast routing

Bellman-Ford → Best for negative edges

MST → Best for network design

🔹 2. Complexity vs Scalability

BFS/DFS:

Good for sparse graphs

Slower in dense graphs

Bellman-Ford:

Very slow for large graphs

Not scalable

🔹 3. Infrastructure Impact (MST)

Prim → faster in dense networks

Kruskal → cheaper in sparse networks

Choice affects:

Cost

Performance

🔹 4. Edge Case Handling

Problem:

Negative weight cycles

Solution:

Use Bellman-Ford

Detect cycle in final iteration

🔹 5. Extensibility

Example Problem:

Social Media Recommendation System

Algorithm:

BFS (friend suggestions)

Graph traversal

Code

[37]:
```python
#GRAPH
graph = {
    0: [(1, 2), (3, 6)],
    1: [(0, 2), (2, 3), (3, 8), (4, 5)],
    2: [(1, 3), (4, 7)],
    3: [(0, 6), (1, 8)],
    4: [(1, 5), (2, 7)]
}#Prim's Algorithm
import heapq

def prim(graph, start=0):
    visited = set()
    min_heap = [(0, start)]
    total_cost = 0

    while min_heap:
        weight, node = heapq.heappop(min_heap)

        if node not in visited:
            visited.add(node)
            total_cost += weight

            for neighbor, w in graph[node]:
                if neighbor not in visited:
                    heapq.heappush(min_heap, (w, neighbor))

    return total_cost
```

```python
    edges.sort(key=lambda x: x[2])

    parent = {}
    rank = {}

    for node in graph:
        parent[node] = node
        rank[node] = 0

    total_cost = 0

    for u, v, w in edges:
        if find(parent, u) != find(parent, v):
            union(parent, rank, u, v)
            total_cost += w

    return total_cost
    #Execution + Time Comparison
import time

# Prim
start = time.perf_counter()
prim_cost = prim(graph)
prim_time = time.perf_counter() - start
                                                    #Prim → O((V + E) Log V)
# Kruskal                                           #Kruskal → O(E Log E)
start = time.perf_counter()
kruskal_cost = kruskal(graph)
kruskal_time = time.perf_counter() - start
```

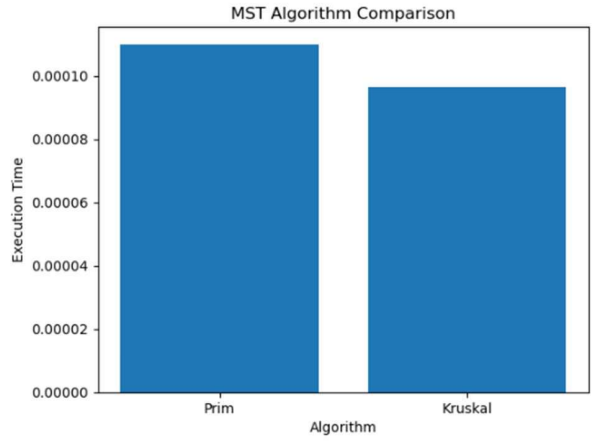
[39]:
```python
#PLOT Comparison
import matplotlib.pyplot as plt

methods = ["Prim", "Kruskal"]
times = [prim_time, kruskal_time]
plt.bar(methods, times)
plt.xlabel("Algorithm")
plt.ylabel("Execution Time")
plt.title("MST Algorithm Comparison")#Execution + Time Comparison
import time
```

```python
#Kruskal's Algorithm
def find(parent, i):
    if parent[i] != i:
        parent[i] = find(parent, parent[i])
    return parent[i]

def union(parent, rank, x, y):
    root_x = find(parent, x)
    root_y = find(parent, y)

    if rank[root_x] < rank[root_y]:
        parent[root_x] = root_y
    elif rank[root_x] > rank[root_y]:
        parent[root_y] = root_x
    else:
        parent[root_y] = root_x
        rank[root_x] += 1


def kruskal(graph):
    edges = []

    for u in graph:
        for v, w in graph[u]:
            if (v, u, w) not in edges:
                edges.append((u, v, w))

    edges.sort(key=lambda x: x[2])
```

```python
# Prim
start = time.perf_counter()
prim_cost = prim(graph)
prim_time = time.perf_counter() - start
                                                    #Prim → O((V + E) Log V)
# Kruskal                                           #Kruskal → O(E Log E)
start = time.perf_counter()
kruskal_cost = kruskal(graph)
kruskal_time = time.perf_counter() - start

print("Prim MST Cost:", prim_cost)
print("Kruskal MST Cost:", kruskal_cost)

print("Prim Time:", prim_time)
print("Kruskal Time:", kruskal_time)
```

Output

```
Prim MST Cost: 16
Kruskal MST Cost: 16
Prim Time: 0.00010999999994965037
Kruskal Time: 9.649999992689118e-05
```



[ ]:

GITHUB LINK = mnjeetydv2923-ops/Graph-algo-mini-project-manjeet: Implementation of core graph algorithms (BFS, DFS, Topological Sort, Shortest Path, and Minimum Spanning Tree) with real-world problem applications, performance analysis, and visualization using Python